Découvrez le fonctionnement de MongoDB

Mise en place d'une base de données MongoDB

MongoDB est une base de données NoSQL orientée documents. Comme nous le verrons, l'ensemble du système tourne autour de cette gestion de documents, y compris le langage d'interrogation, ce qui en fait son point fort. Nous allons nous attaquer dès maintenant à la mise en place d'un serveur Mongo et comment intégrer vos données dans cet environnement.

Installation

Pour bien fonctionner, une base *MongoDB* a besoin de trois choses :

- L'installation du serveur, que vous pouvez télécharger ici (version 3.4.7)

L'installation sous *Windows* est assez simple (appuyez sur suivant).

Sous Mac, décompressez l'archive et déplacez le répertoire dans vos applications "/Applications"

Sous Debian/Ubuntu, utilisez la commande "*sudo apt install mongodb*".

Le serveur sera alors installé dans le répertoire que je nommerai : **$MONGO**.

Sous *Windows* : C:\Program Files\MongoDB\Server\3.4\
Sous *Linux* : /opt/local/bin/
Sous *Mac* : /Applications

- La création d'un répertoire pour stocker les données. Par exemple : C:\data\db (par défaut)

Sous *Windows*, il faut ouvrir l'explorateur de fichiers, aller à la racine "C:\", puis créer le répertoire "data". Ensuite, aller dans ce répertoire et créer le répertoire "db".

Sous *Linux ou Mac*, il faut créer le répertoire data à la racine avec "sudo mkdir ~/data". Vous donnez les droits (ex. utilisateur 'toto') : "sudo chown toto ~/data". Et enfin, créer le répertoire db : "mkdir ~/data/db".

Sous *Mac*, décompressez l'archive et déplacez le répertoire dans vos applications "/Applications"

Il est possible de créer un autre répertoire dédié "*$REP*" pour les données de MongoDB. Dans ce cas, le lancement du serveur ci-dessous devra utiliser le paramètre --dbpath $REP. C'est le cas sous Mac comme effectué ci-dessus, cela donne : "*mongod --dbpath ~/data/db*"

- Le lancement du serveur, avec l'exécutable mongod (disponible sur $MONGO/bin)

Si tout se passe bien, le serveur devrait tourner comme dans le screenshot ci-dessous (ne pas fermer la fenêtre sous peine d'éteindre la base de données). Le serveur attend une connexion sur le port 27017.

```
2017-06-20T23:48:42.486+0200 I CONTROL  [initandlisten] MongoDB starting : pid=90067 port=27017 dbpath=/data/db 64-bit host=MacBook-Air.local
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] db version v3.4.3
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] git version: f07437fb5a6cca07c10bafa78365456eb1d6d5e1
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] OpenSSL version: OpenSSL 1.0.2k  26 Jan 2017
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] allocator: system
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] modules: none
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] build environment:
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten]     distarch: x86_64
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten]     target_arch: x86_64
2017-06-20T23:48:42.488+0200 I CONTROL  [initandlisten] options: {}
2017-06-20T23:48:42.493+0200 I -        [initandlisten] Detected data files in /data/db created by the 'mmapv1' storage engine, so setting the active storage engine to 'mma
pv1'.
2017-06-20T23:48:42.508+0200 I JOURNAL  [initandlisten] journal dir=/data/db/journal
2017-06-20T23:48:42.509+0200 I JOURNAL  [initandlisten] recover : no journal files present, no recovery needed
2017-06-20T23:48:42.528+0200 I JOURNAL  [durability] Durability thread started
2017-06-20T23:48:42.529+0200 I JOURNAL  [journal writer] Journal writer thread started
2017-06-20T23:48:42.529+0200 I CONTROL  [initandlisten]
2017-06-20T23:48:42.529+0200 I CONTROL  [initandlisten] ** WARNING: Access control is not enabled for the database.
2017-06-20T23:48:42.529+0200 I CONTROL  [initandlisten] **          Read and write access to data and configuration is unrestricted.
2017-06-20T23:48:42.529+0200 I CONTROL  [initandlisten]
2017-06-20T23:48:42.529+0200 I CONTROL  [initandlisten]
2017-06-20T23:48:42.529+0200 I CONTROL  [initandlisten] ** WARNING: soft rlimits too low. Number of files is 256, should be at least 1000
2017-06-20T23:48:42.868+0200 I FTDC     [initandlisten] Initializing full-time diagnostic data capture with directory '/data/db/diagnostic.data'
2017-06-20T23:48:42.870+0200 I NETWORK  [thread1] waiting for connections on port 27017
```

Log de connexion du serveur mongod

Interface utilisateur

Maintenant que le serveur tourne, on peut s'attaquer à son administration avec une interface graphique. Nous prendrons Robo3T version 1.1 (anciennement *robomongo*) qui est largement utilisé.

L'installation est assez simple sur Windows et Mac, il suffit de suivre les instructions.

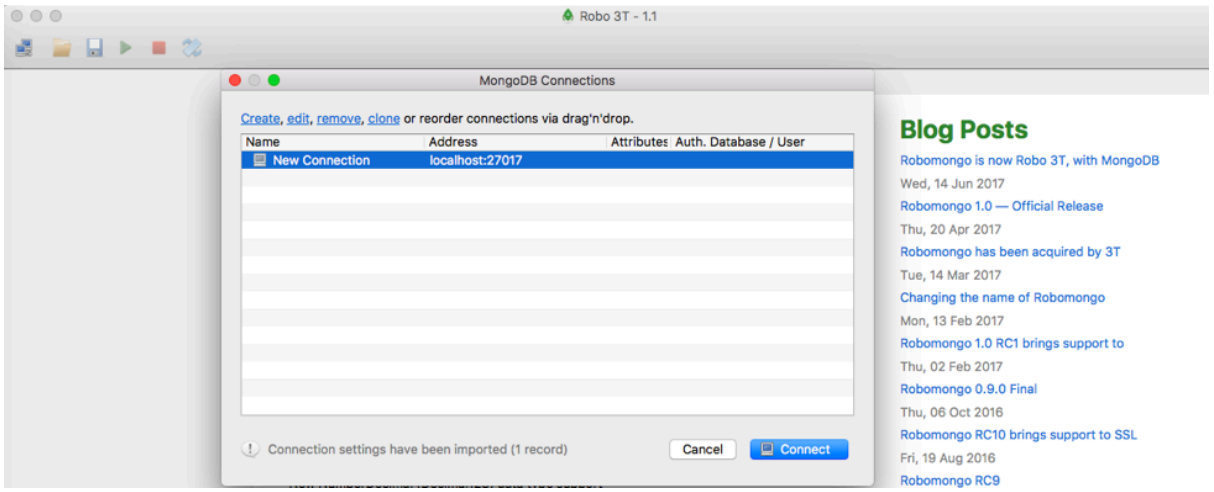
Sous *Windows*, l'application est installée par défaut dans le répertoire *C:\Program Files\3T\Robo3T*

Sous *Mac*, il suffit de placer Robo3T dans le répertoire "Applications"

Sous *Linux*, le fichier d'installation doit être décompressé (*tar xzvf robo3t-1.1.1-linux-x86_64-c93c6b0.tar.gz).* Puis, vous pouvez exécuter le binaire "bin/robo3t"

Il peut être demandé la plateforme "Qt" avec le plugin "xcb". Dans ce cas, suivez la procédure indiqué **ici**.

Après lancement de l'application, cliquez sur "*create*" comme dans l'image ci-dessous pour créer une nouvelle connexion à MongoDB (avec "**localhost**" et le port "**27017**" par défaut).



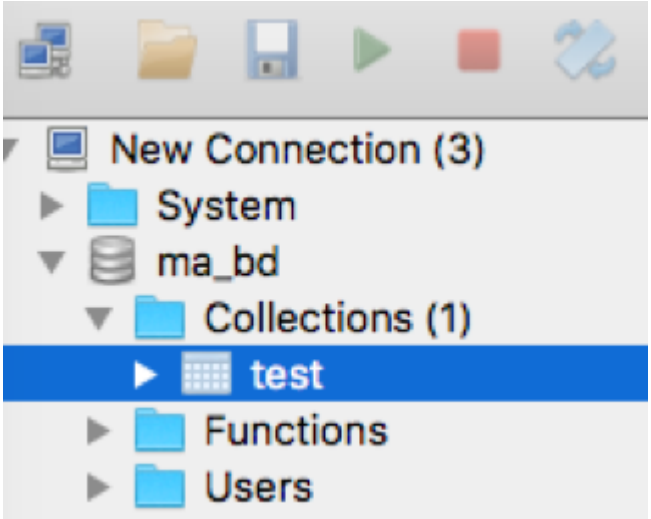
Création d'une connexion sous Robo3T

Pour une utilisation plus intensive, je recommanderai de prendre une licence pour **Studio 3T** (Trial de 2 semaines). Il facilite les imports de données, la lecture de schéma, la conception de requêtes...

Création d'une collection

Une collection c'est l'équivalent d'une table pour une base de données relationnelle. Mais dans notre cas, une collection n'a pas de schéma donnant la structure d'une collection. Comme nous le verrons par la suite un document n'a pas de structure fixe, de fait, une base de données gère une 'collection' de documents, et non une table.

Créons tout d'abord une base de données 'ma_bd' (bouton droit sur la connexion "Create database"), puis sur les collections de cette base, créer une collection "test" (bouton droit sur "Collections(0)")



Création d'une collection sous Robo3T

Modélisation de documents

Documents JSON

Maintenant que l'environnement est prêt, une question se pose : à quoi ressemblent mes données ? À des documents JSON. Le modèle est très simple :

1. Tout est clé/valeur : **"clé" : "valeur"**
2. Un document est encapsulé dans des accolades **{...}**, pouvant contenir des listes de clés/valeurs
3. Une valeur peut être un type scalaire (entier, nombre, texte, booléen, null), des listes de valeurs **[...]**, ou des documents imbriqués

On peut donc insérer un document dans notre collection "test" (double click sur la collection, champ de texte en noir pour exécuter une requête) :

```python
db.test.save (
{
 "cours" : "NoSQL",
 "chapitres" : ["familles", "CAP", "sharding", "choix"],
```

```
 "auteur" : {

   "nom" : "Travers",

   "prenom" : "Nicolas"

 }

} )
```

Il faut savoir que MongoDB ne gère aucun schéma pour les collections. Donc pas de structure fixe ni de typage, ce qui fait la force des SGBDR. Du coup, il est difficile lors de l'interrogation de connaître le contenu de la base de données. Le logiciel **Studio3T** permet d'extraire des statistiques sur la structure des documents. Ce qui permet d'en avoir une vision globale.

Relationnel vers JSON - comment faire ?

Cela semble simple, c'est à s'y méprendre ! En effet, les données utilisées de manière classiques sont en relationnel (Exportation en CSV pour faire simple). Mais voilà, nous devons faire face à un gros problème dans les bases NoSQL : Il faut proscrire les jointures !

De fait, si vous avez un SGBDR avec deux tables, vous ne pourrez pas faire de requêtes de jointure (ou alors, ça vous coûtera horriblement cher). Il faut dans ce cas trouver une solution pour fusionner les deux tables pour n'en produire qu'une seule en sortie. C'est ce qu'on appelle la dénormalisation.

Reprenons notre exemple précédent, mais avec un schéma un peu plus complexe (schéma de gauche). Une personne peut avoir plusieurs domaine d'expertise, des emplois successifs, et une habitation :

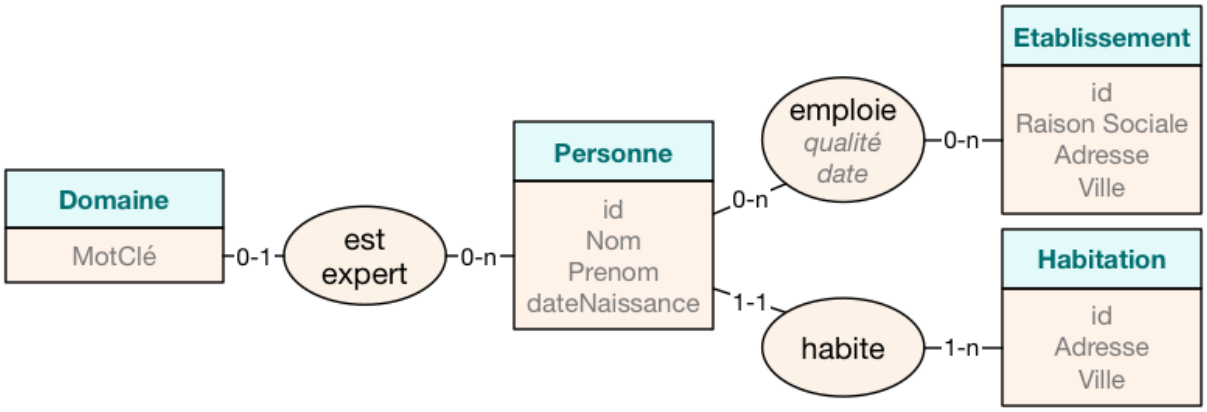


Schéma Entité/Association original

Si je dois intégrer une base de données avec une collection par entité (rectangle) et association (ovales), le nombre de jointures pour les requêtes sur la base NoSQL risque de faire exploser le système. Du coup, des fusions sont nécessaires pour réduire le coût des requêtes. Mais quelles tables doit-on imbriquer ? Dans quel sens le faire ?

Dénormalisation du schéma

Voici quelques étapes de modélisation qui vont vous permettre de produire des documents JSON qui répondront à votre demande, tout en minimisant les problèmes de jointures et d'incohérences :

- **Des données fréquemment interrogées conjointement.**
  Par exemple, les requêtes demandent fréquemment le lieu d'habitation d'une personne. De fait, la jointure devient coûteuse. Accessoirement, cette information étant peu mise à jour, cela pose peu de problèmes. Résultat, l'entité 'Habitation' et l'association 'habite' sont intégrés à l'entité Personne. Habitation devient un document imbriqué à l'intérieur de Personne, représenté par : "{habite}"
- **Toutes les données d'une entité sont indépendantes.**
  Prenons l'exemple des domaines d'expertise d'une personne, ils sont indépendants des domaines d'une autre personne. De fait, rapatrier les données de cette entité n'impacte aucune autre instance de Personne. Ainsi, la liste des domaines est importé dans Personne et représenté par : "[domaines]"
- **Une association avec des relations 1-n des deux côtés.**
  Cette fois-ci, c'est plus délicat pour l'entité Etablissement. Une personne peut avoir plusieurs emplois et un employeur, plusieurs employés. De fait, une imbrication de l'employeur dans Personne peut avoir de gros impacts sur les mises à jour (tous les employés à mettre à jour !). Il est donc peu recommandé d'effectuer une fusion complète. Pour cela, seule l'association est imbriquée sous forme d'une liste de documents, intégrant les attributs

(qualité et date), ainsi qu'une référence vers l'employeur. Ainsi : "[{emploie+ref}]"

- **Même taux de mises à jour.**
  Dans le cas des emplois d'une personne, là également nous pourrions effectuer une fusion de l'association "emploie". En effet, le taux de mises à jour des emplois est équivalent à celui de la Personne, de fait, sans incidence sur les problèmes de cohérence de données.

Ces étapes de modélisation sont un résumé de plusieurs articles de recherches en Système d'Information. Si vous souhaitez en lire d'avantage : Query-Driven [**Chebotko15**], MDA-NoSQL [**Abdelhedi et al. 17**], Document-oriented NoSQL modelling [**Mason15**].

Au vu de ces critères, voici le schéma qui pourrait être obtenu par fusion successives :

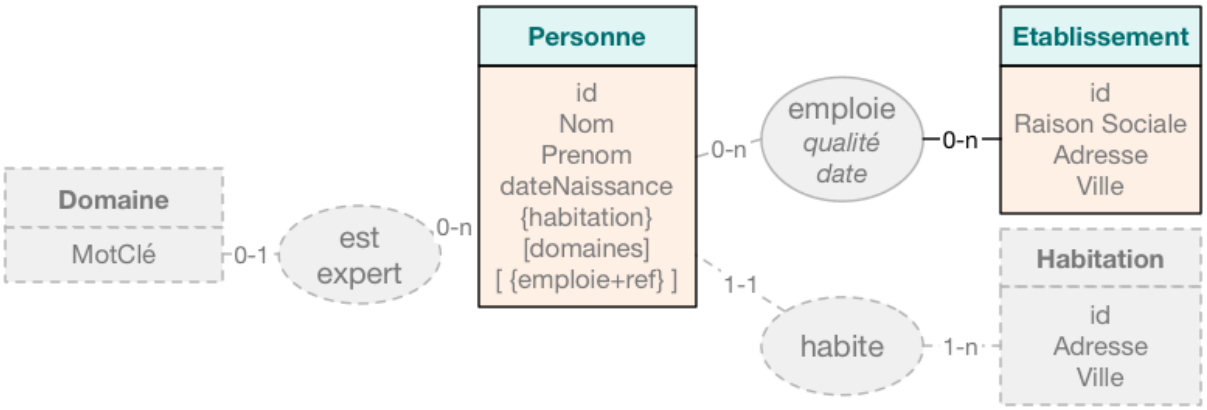


Schéma dénormalisé

Les listes sont représentées par des crochets et les imbrications par des accolades. Nous pourrons remarquer que les établissements sont stockés dans une association à part, mais que la référence est gardée dans une liste intégrée à l'entité "Personne". Voici un exemple d'instance de cette représentation. Vous pouvez remarquer que les établissements sont référencés et des informations peuvent ne pas être renseignées.

Unset

```
{

 "_id" : 1,    "nom" : "Travers",   "prenom" : "Nicolas",
```

```
 "domaines" : ["SGBD", "NoSQL", "RI", "XML"],

 "emplois" : [

   {"id_etablissement" : "100", "qualité" : "Maître de Conférences",

      "date" : "01/09/2007"},

   {"id_etablissement" : "101", "qualité" : "Vacataire",

      "date" : "01/09/2012"}

 ],

 "Habite" : {"adresse" : "292 rue Saint Martin", "ville" : "Paris"}

}
```

Maintenant, nous sommes prêts pour concevoir des bases NoSQL orientées documents avec des fusions. Ne reste plus qu'à faire la conversion de données relationnelles vers ces documents. Pour cela, il est possible de créer un ETL (Extract-Transform-Load) avec votre langage de programmation préféré qui va, à la volée, chercher les informations nécessaires pour produire des données correspondant au schéma. Une seconde possibilité est d'utiliser l'outil OpenRefine pour nettoyer et définir un format de sortie JSON. La dernière possibilité est d'intégrer chaque fichier dans MongoDB (chacun une collection distincte), puis d'utiliser les opérateurs $lookup et $redact pour produire la sortie souhaitée

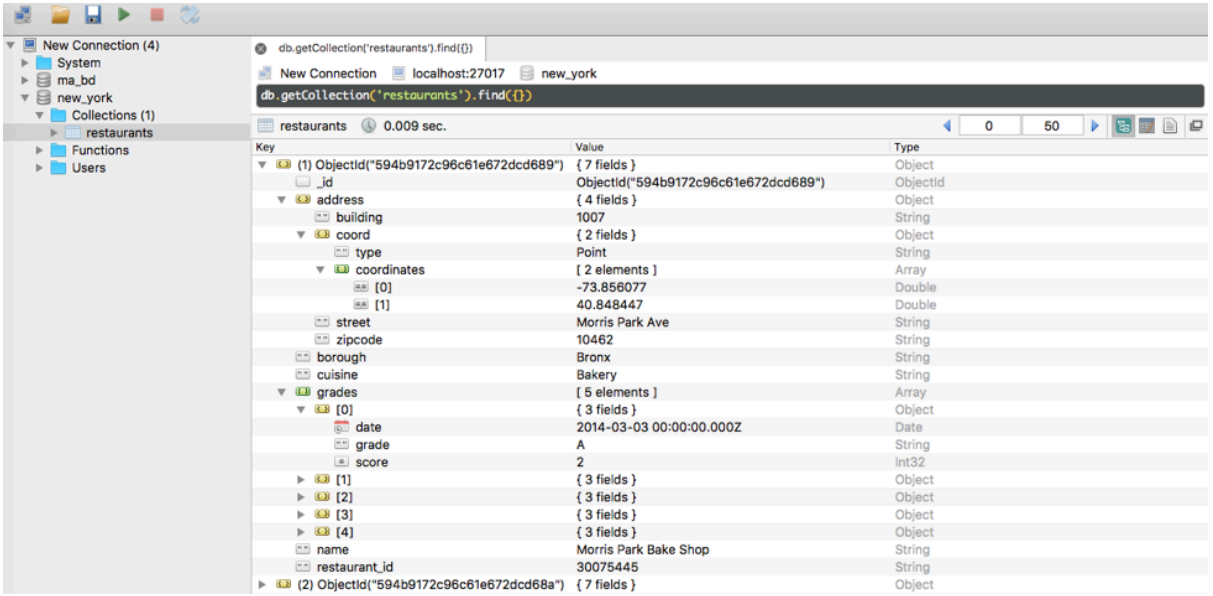
Importer les données

Nous allons maintenant importer un jeu de données déjà formaté à notre base de données. Un jeu de données OpenData est disponible sur des restaurants de New York produits par la mairie sur les résultats des inspections. Pour l'importation :

1. Téléchargez l'archive suivante : restaurants.zip
2. Décompresser l'archive (j'appellerai le répertoire utilisé **$RESTAURANT**)
3. Nous allons créer une base de données "**new_york**" (paramètre *--db*) et une collection "**restaurants**" (paramètre *--collection*). Attention, il ne faut pas de majuscules !
4. Dans une console (Windows : invite de commande, Linux : Shell/Konsole), aller dans le répertoire *$MONGO/bin*
5. Exécutez la commande suivante :

```
$MONGO/bin/mongoimport --db new_york --collection restaurants
$RESTAURANT/restaurants.json
```

Ca y est, vous avez une base de données de 25 357 restaurants que vous pouvez consulter directement avec Robo3T.



Screenshot de la collection "restaurants" sous Robo3T

Nous allons maintenant pouvoir interroger cette collection avec le langage proposé par MongoDB.

Interrogez vos données avec MongoDB

Le langage de MongoDB repose sur Javascript, ce qui facilite l'utilisation de JSon, des appels de fonctions, la création de librairies ou la définition de programmes Map/Reduce.

Toute commande sur la collection restaurants utilise le préfixe : "**db.restaurants**".

Il suffira d'y associer la fonction souhaitée pour avoir un résultat.

Avant de commencer, il faut voir à quoi ressemble un document de notre collection de restaurants. Utilisons la fonction "*findOne()*".

```
Unset

db.restaurants.findOne()





{

   "_id" : ObjectId("594b9172c96c61e672dcd689"),

   "restaurant_id" : "30075445",

   "name" : "Morris Park Bake Shop",

   "borough" : "Bronx",

   "cuisine" : "Bakery",

   "address" : {

      "building" : "1007",

      "coord" :{"type":"Point","coordinates":[-73.856077,40.848447]},
```

```
      "street" : "Morris Park Ave",

      "zipcode" : "10462"

   },

   "grades" : [

      {"date" : ISODate("2014-03-03T00:00:00.000Z"),"grade" : "A","score" :
2},

      {"date" : ISODate("2013-09-11T00:00:00.000Z"),"grade" : "A","score" :
6},

      {"date" : ISODate("2013-01-24T00:00:00.000Z"),"grade" : "A","score" :
10},

      {"date" : ISODate("2011-11-23T00:00:00.000Z"),"grade" : "A","score" :
9},

      {"date" : ISODate("2011-03-10T00:00:00.000Z"),"grade" : "B","score" :
14}

   ]

}
```

Chaque restaurant a un nom, un quartier (*borough*), le type de cuisine, une adresse (document imbriqué, avec coordonnées GPS, rue et code postale), et un ensemble de notes (résultats des inspections).

Filtrer et Projeter les données avec un motif JSon

Le point fondamental à retenir pour le langage de MongoDB est le "clé/valeur", avec des définitions de motifs en documents **JSON**. Ainsi, chaque requête sera un document, avec des clés et des valeurs. Le deuxième point est la structure d'un document. Chaque document est traité indépendamment, comme en relationnel, mais les documents contiennent maintenant des imbrications et des listes, ce qui n'est pas habituel. Nous allons en illustrer les conséquences à partir d'exemples.

Filtrage

Commençons par un type simple de requête : le filtrage. Nous allons pour cela utiliser la fonction "find" à appliquer directement à la collection. L'exemple ci-dessous récupère tous les restaurants dans le quartier (borough) de Brooklyn.

Unset

```
db.restaurants.find( { "borough" : "Brooklyn" } )
```

6 085 restaurants sont retournés.

Pour compter, il suffit d'ajouter la fonction "count"

Unset

```
db.restaurants.find( { "borough" : "Brooklyn" } ).count()
```

En effet, nous sommes en *Javascript* : le résultat d'un *find* est une liste dont on peut compter les documents avec la méthode de liste "count"

Maintenant, nous cherchons parmi ces restaurants ceux qui font de la cuisine italienne. Pour combiner deux "clés/valeurs", il suffit de faire le document motif donnant quels paires clés/valeurs sont recherchés :

```
Unset

db.restaurants.find(

   { "borough" : "Brooklyn",

    "cuisine" : "Italian" }

)
```

Ensuite, on va chercher les restaurants présents sur la 5° Avenue. La clé "street" est imbriquée dans l'adresse ; comment filtrer une clé imbriquée (obtenue après fusion) ? Il faut utiliser les deux clés, en utilisant un point "." comme si c'était un objet.

```
Unset

db.restaurants.find(

   { "borough" : "Brooklyn",

    "cuisine" : "Italian",

    "address.street" : "5 Avenue" }

)
```

Pourquoi ne pas chercher le mot "pizza" dans le nom du restaurant ? Pour cela, il faut utiliser les expressions régulières avec "/xxx/i" (le i pour "Insensible à la casse" - majuscule/minuscule).

```
Unset

db.restaurants.find(

   { "borough" : "Brooklyn",

    "cuisine" : "Italian",

    "address.street" : "5 Avenue",

    "name" : /pizza/i }

)
```

Il ne reste maintenant que 2 restaurants Italiens dans le quartier de Brooklyn dans la 5° Avenue, dont le nom contient le mot "pizza".

Projection

Et si nous ne gardions dans le résultat que le nom ? C'est ce que l'on appelle une projection. Un deuxième paramètre (optionnel) de la fonction *find* permet de choisir les clés à retourner dans le résultat. Pour cela, il faut mettre la clé, et la valeur "1" pour projeter la valeur (on reste dans le concept "clé/valeur").

```
Unset

db.getCollection('restaurants').find(

   {"borough":"Brooklyn",
```

```
   "cuisine":"Italian",

   "name":/pizza/i,

   "address.street" : "5 Avenue"},

   {"name":1}

)



{"_id" : ObjectId("594b9173c96c61e672dd074b"), "name" : "Joe'S Pizza"}

{"_id" : ObjectId("594b9173c96c61e672dd1c16"), "name" : "Gina'S Pizzaeria/
Deli"}
```

L'identifiant du document "_id" est automatiquement projeté, si vous souhaitez le faire disparaître il suffit de rajouter la valeur "0" : {"name":1, "_id":0}

Et si nous regardions les résultats des inspections des commissions d'hygiène ? Il faut pour cela regarder la valeur de la clé "*grades.score*".

Unset

```
db.getCollection('restaurants').find(
```

```
   {"borough":"Brooklyn",

   "cuisine":"Italian",

   "name":/pizza/i,

   "address.street" : "5 Avenue"},

   {"name" : 1,

   "grades.score" : 1}

)



{

   "name" : "Joe'S Pizza",

   "grades" : [

      {"score" : 12},

      {"score" : 13},
```

```
      {"score" : 42},

      {"score" : 25},

      {"score" : 19},

      {"score" : 40},

      {"score" : 22}

   ]

}

{

   "name" : "Gina'S Pizzaeria/ Deli",

   "grades" : [

      {"score" : 12},

      {"score" : 23},

      {"score" : 13}
```

```
    ]

}
```

Ce qui est intéressant est de voir que le score est retrouvé dans les documents imbriqués de la liste "grades", et que tous les scores sont projetés.

À noter qu'un score bas est un bon score pour une inspection aux États-Unis

Filtrage avec des opérations

Le filtrage par valeur exacte n'est pas suffisant pour exprimer tout ce que l'on souhaiterait trouver dans la collection. Je cherche maintenant les noms et scores des restaurants de Manhattan ayant un score inférieur à 10. On peut utiliser des opérateurs arithmétiques sur les clés (valeur numérique). Pour cela, il sera préfixé d'un dollar **$**. "Plus petit que" en anglais se dit "*less than*", l'opérateur est donc **$lt**. Il faut y associer la valeur souhaitée : **"grades.score" : {"$lt" : 10}**. Vous trouverez les détails ici.

L'opérateur est **toujours** imbriqué dans un document. Le concept "clé/valeur" doit être respecté. Voici les opérateurs disponibles avec des exemples associés :

$gt, $gte	>, ≥	Plus grand que (*greater than*)	"a" : {"$gt" : 10}
$lt, $lte	<, ≤	Plus petit que (*less than*)	"a" : {"$lt" : 10}
$ne	≠	Différent de (*not equal*)	"a" : {"$ne" : 10}
$in, $nin	∈, ∉	Fait parti de (ou ne doit pas)	"a" : {"$in" : [10, 12, 15, 18] }
$or	□	OU logique	"a" : {“$or” : [{"$gt" : 10}, {“$lt” : 5}] }

$and	□	ET logique	"a" : {“$and” : [{"$lt" : 10}, {“$gt” : 5}] }
$not	¬	Négation	“a" : {“$not” : {"$lt" : 10} }
$exists	∃	La clé existe dans le document	“a” : {“$exists” : 1}
$size		test sur la taille d'une liste (uniquement par égalité)	“a” : {“$size” : 5}

La requête donne ceci :

```
Unset

db.getCollection('restaurants').find(

  {"borough":"Manhattan",

  "grades.score":{$lt : 10}

  },

  {"name":1,"grades.score":1, "_id":0})



{

  "name" : "Dj Reynolds Pub And Restaurant"
```

```
  "grades" : [

    {"score" : 2},

    {"score" : 11},

    {"score" : 12},

    {"score" : 12}

  ]

}
```

Ce qui est déroutant dans ce résultat, c'est le fait que l'on trouve des scores supérieurs à 10 !

Nous ne sommes plus en relationnel, ainsi, l'opération **"grades.score" : {"$lt" : 10}**, veut dire :

Est-ce que la liste "grades" contient un score (**au moins**) avec une valeur inférieure à 10 ?

Et en effet, il y a un score à "2" respectant la question.

Si l'on souhaite ne récupérer que ceux qui n'ont pas de score supérieur à 10, il faut alors combiner la première opération avec une négation de la condition "

≥10

≥10 ". La condition est alors vérifiée sur chaque élément de la liste.

Unset

```
db.getCollection('restaurants').find(

  {"borough":"Manhattan",

  "grades.score":{

     $lt:10,

     $not:{$gte:10}

  }

  },

  {"name":1,"grades.score":1, "_id":0})



{

  "name" : "1 East 66Th Street Kitchen",

  "grades" : [
```

```
      {"score" : 3},

      {"score" : 4},

      {"score" : 6},

      {"score" : 0}

   ]

}
```

On pourrait même corser la chose en cherchant les restaurants qui ont un grade ‘C’ avec un score inférieur à 40.

```
Unset
db.restaurants.find({

   "grades.grade" : "C",

   "grades.score" : {$lt : 30}

  },
```

```
  {"Grades.grade":1, "grades.score":1}

);



{

   "_id" : ObjectId("594b9172c96c61e672dcd695"),

   "grades" : [

     {"grade" : "B","score" : 21},

     {"grade" : "A","score" : 7},

     {"grade" : "C","score" : 56},

     {"grade" : "B","score" : 27},

     {"grade" : "B","score" : 27}

   ]

}
```

```
{
  "_id" : ObjectId("594b9172c96c61e672dcd6bc"),
  "grades" : [
    {"grade" : "A","score" : 9},
    {"grade" : "A","score" : 10},
    {"grade" : "A","score" : 9},
    {"grade" : "C","score" : 32}
  ]
}
```

Le résultat est pour le moins étrange concernant le premier document car il y a bien un grade C, mais avec un score de 56 ! Si l'on regarde la requête de plus près, ce n'est pas étonnant. Y a-t-il un grade 'C' ? oui. Y a-t-il un score inférieur à 40 ? oui… Nous avons oublié de préciser qu'il fallait que ce soit vérifié sur chaque élément ! La vérification se fait sur l'intérieur de la liste, pas sur chaque élément de la liste. Pour cela, un opérateur permet de faire une vérification instance par instance : **$elemMatch**.

```
Unset
db.restaurants.find({
```

```
   "grades" : {

        $elemMatch : {

           "grade" : "C",

           "score" : {$lt :40}

        }

   }

 },

 {"grades.grade" : 1,"grades.score" : 1}

);



{

   "_id" : ObjectId("594b9172c96c61e672dcd6bc"),

   "grades" : [
```

```
        {"grade" : "A","score" : 9},

        {"grade" : "A","score" : 10},

        {"grade" : "A","score" : 9},

        {"grade" : "C","score" : 32}

   ]

}
```

Cette fois-ci le résultat est bon.

Une dernière pour finir, je voudrais maintenant les noms et quartiers des restaurants dont la dernière inspection (la plus récente, donc la première de la liste) a donné un grade ‘C’. Il faut donc chercher dans le premier élément de la liste. Pour cela il est possible de rajouter l’indice recherché (indice 0) dans la clé.

```
Unset
db.restaurants.find({

   "grades.0.grade":"C"

 },
```

```
 {"name":1, "borough":1, "_id":0}

);

{"borough" : "Queens","name" : "Mcdonald'S"}

{"borough" : "Queens","name" : "Nueva Villa China Restaurant"}

{"borough" : "Queens","name" : "Tequilla Sunrise"}

{"borough" : "Manhattan","name" : "Dinastia China"}
```

Distinct

Au fait, quels sont les différents quartiers de New York ? Pour cela, on peut utiliser la fonction "distinct()" avec la clé recherchée pour avoir une liste de valeur :

```
Unset
db.restaurants.distinct("borough")

[

   "Bronx",
```

```
  "Brooklyn",
  "Manhattan",
  "Queens",
  "Staten Island",
  "Missing"
]
```

Et si nous le faisions sur une liste de valeurs comme les grades donnés par les inspecteurs ?

```
Unset
db.restaurants.distinct("grades.grade");
[
  "A",
```

```
    "B",

    "Z",

    "C",

    "P",

    "Not Yet Graded"

  ]
```

La fonction distincte va donc chercher les instances de la liste pour en extraire chaque grade et produire une liste distincte (plutôt que de faire une liste de liste distincte).

Créer une séquence d'opérations avec "*aggregate*"

Et si maintenant, nous regardions des opérations plus complexes pour bien manipuler nos données ? Pour cela, la fonction "**aggregate ()**" permet de spécifier des chaînes d'opérations, appelées pipeline d'agrégation.

Cette fonction aggregate prend une liste d'opérateurs en paramètre. Il existe plusieurs types d'opérateurs de manipulation de données. Nous allons nous concentrer par la suite sur les principaux :

- **{$match : {} }** : C'est le plus simple, il correspond au premier paramètre de la requête find que nous avons fait jusqu'ici. Il permet donc de filtrer le contenu d'une collection.
- **{$project : {} }** : C'est le second paramètre du find. Il donne le format de sortie des documents (projection). Il peut par ailleurs être utilisé pour changer le format d'origine.
- **{$sort : {} }** : Trier le résultat final sur les valeurs d'une clé choisi.

- **{$group : {} }** : C’est l’opération d’agrégation. Il va permettre de grouper les documents par valeur, et appliquer des fonctions d’agrégat. La sortie est une nouvelle collection avec les résultats de l’agrégation.
- **{$unwind : {} }** : Cet opérateur prend une liste de valeur et produit pour chaque élément de la liste un nouveau document en sortie avec cet élément. Il pourrait correspondre à une jointure, à ceci près que celle-ci ne filtre pas les données d’origine, juste un complément qui est imbriqué dans le document. On pourrait le comparer à une jointure externe avec imbrication et listes.

Vous aurez remarqué que chaque opérateur est imbriqué dans un document, et la valeur est elle-même un autre document. C’est la structure imposée pour la définition de l’opérateur. Pour illustrer un aggregate, je vais reprendre notre dernière requête *find* :

```
Unset

db.restaurants.aggregate( [

   { $match : {

      "grades.0.grade":"C"

   }},

   { $project : {

      "name":1, "borough":1, "_id":0

   }}

] )
```

Le résultat est identique, toutefois, c’est un peu plus lourd à écrire. On identifie bien chaque opérateur, avec leur définition.

Rappelez-vous que l'interpréteur mongodb utilise le langage Javascript. Chaque opérateur peut alors être défini par une variable que nous pouvons utiliser directement :

```
Unset

varMatch = { $match : { "grades.0.grade":"C"} };

varProject = { $project : {"name":1, "borough":1, "_id":0}};

db.restaurants.aggregate( [ varMatch, varProject ] );
```

Nous utiliserons ces variables par la suite pour alléger le code en ré-utilisant des variables.

Tri

Trions maintenant le résultat par nom de restaurant par ordre croissant (croissant : 1, décroissant : -1) :

```
Unset

varSort = { $sort : {"name":1} };
```

```
db.restaurants.aggregate( [ varMatch, varProject, varSort ] );
```

Groupement simple

Comptons maintenant le nombre de ces restaurants (premier rang ayant pour valeur C). Pour cela, il faut définir un opérateur $group. Celui-ci doit contenir obligatoirement une clé de groupement (_id), puis une clé (total) à laquelle on associe la fonction d'agrégation ($sum) :

```
Unset

varGroup = { $group : {"_id" : null, "total" : {$sum : 1} } };

db.restaurants.aggregate( [ varMatch, varGroup ] );

{"_id" : null, "total" : 220}
```

Cette requête est équivalente à :

```
Unset

db.restaurants.count({"grades.0.grade":"C"})
```

```
db.restaurants.find({"grades.0.grade":"C"}).count()
```

Mais si vous souhaitez faire un pipeline d'agrégation, vous serez obligé d'utilisé la fonction "aggregate".

Ici, pas de valeur de groupement demandé (on compte simplement). Nous avons choisi de mettre la valeur *null*, on aurait pu mettre "toto", cela fonctionne également. La clé "total" est créée dans le résultat et va effectuer la "somme des 1" pour chaque document source. Faire une somme de 1 est équivalent à compter le nombre d'éléments.

Groupement par valeur

Comptons maintenant par quartier le nombre de ces restaurants. Il faut dans ce cas changer la clé de groupement en y mettant la valeur de la clé "borough". Mais si l'on essaye ceci :

Unset

```
varGroup2 = { $group : {"_id" : "borough", "total" : {$sum : 1} } };
```

```
db.restaurants.aggregate( [ varMatch, varGroup2 ] );
```

```
{"_id" : "borough", "total" : 220}
```

La valeur de la clé de groupement prend une unique valeur "*borough*". Le problème vient du fait que nous n'avons pas pris la valeur, mais seulement précisé un texte ! Pour prendre la valeur, il faut préfixer la clé par un dollar "$borough", cela va, lors de l'exécution, indiquer qu'il faut utiliser la valeur de la clé pour l'agrégation.

```
Unset

varGroup3 = { $group : {"_id" : "$borough", "total" : {$sum : 1} } };

db.restaurants.aggregate( [ varMatch, varGroup3 ] );

{"_id" : "Bronx",        "total" : 27.0}

{"_id" : "Staten Island",    "total" : 7.0}

{"_id" : "Manhattan",        "total" : 83.0}

{"_id" : "Brooklyn",        "total" : 56.0}

{"_id" : "Queens",        "total" : 47.0}
```

Ce qui fait bien un total de 220 !

Unwind

Maintenant, trouvons le score moyen des restaurants par quartiers, et trions par score décroissant. Pour cela, nous groupons les valeurs par quartier (*$borough*), nous utilisons la fonction d'agrégat "*$avg*" pour la moyenne (*average*), puis nous trions sur la nouvelle clé produite "$moyenne".

Un problème se pose comment calculer la moyenne sur une "*grades.score*" qui est une liste ? Il faudrait au préalable retirer chaque élément de la liste, puis faire la moyenne sur l'ensemble des valeurs. C'est l'opérateur **$unwind** qui s'en charge, il suffit de lui donner la clé contenant la liste à désimbriquer.

Unset

```
varUnwind = {$unwind : "$grades"}
```

```
varGroup4 = { $group : {"_id" : "$borough", "moyenne" : {$avg :
"$grades.score"} } };
```

```
varSort2 = { $sort : { "moyenne" : -1 } }
```

```
db.restaurants.aggregate( [ varUnwind, varGroup4, varSort2 ] );
```

```
{ "_id" : "Queens", "moyenne" : 11.634865110930088 }
```

```
{ "_id" : "Brooklyn", "moyenne" : 11.447723132969035 }
```

```
{ "_id" : "Manhattan", "moyenne" : 11.41823125728344 }
```

```
{ "_id" : "Staten Island", "moyenne" : 11.370957711442786 }
```

```
{ "_id" : "Bronx", "moyenne" : 11.036186099942562 }
```

```
{ "_id" : "Missing", "moyenne" : 9.632911392405063 }
```

Ce qu'il est important de noter dans cette fonction *aggregate*, c'est que les opérateurs sont composés en séquences : chaque opérateur prend la collection

produite par l'opérateur précédent (et non la collection de départ). Ceci permet de créer de longues chaînes d'opérateurs pour faire des calculs lourds.

Attention ! L'ordre des opérations est très important (projection après les filtrages, tri sur le résultat d'un groupe, filtrage avant ou après *unwind*/*group*…).

Pour des calculs plus compliqués ne pouvant être réalisés avec un *aggregate* (structure conditionnelle, multi-output/fusion, calculs complexes…), MongoDB offre la possibilité de programmer en Map/Reduce (avec javascript). Comme cela sort du cadre de cours sur le langage d'interrogation, vous pourrez trouver la documentation ici et un TP de mise en pratique ici.

Sachez par ailleurs que tous les opérateurs sont programmés en *Map/Reduce* pour faire l'évaluation. Le langage de MongoDB est un langage de plus haut-niveau permettant de faire l'interrogation.

Mettre à jour les données

Une base de données ne serait pas utile si nous ne pouvions faire des mises à jour sur les données. Nous avons déjà vu l'insertion (*save*) dans le chapitre précédent, il faut maintenant regarder les mises à jour (*update*) et les suppressions (*delete*). À chaque modification, il faut préciser le document ciblé (*_id*) et la modification (pour les mises à jour des documents existants).

Update

Pour commencer, nous allons ajouter un commentaire sur un restaurant (opération **$set**) :

Unset

```
db.restaurants.update (
  {"_id" : ObjectId("594b9172c96c61e672dcd689")},
  {$set : {"comment" : "My new comment"}}
);
```

```
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

On peut voir qu'un document a été trouvé (*nMatched*) et modifié (*nModified*). Vous pouvez vérifier s'il a été modifié avec une requête *find*.

Pour supprimer une clé, il suffit de remplacer par l'opérateur **$unset**.

Unset

```
db.restaurants.update (
  {"_id" : ObjectId("594b9172c96c61e672dcd689")},
  {$unset : {"comment" : 1}}
);
```

Faisons une requête avec filtrage pour choisir les restaurants à commenter (faire les documents un par un est fastidieux). Nous allons attribuer un commentaire en fonction des *grades* obtenus. S'il n'a pas eu de note 'C', nous rajouterons le commentaire 'acceptable'.

Unset

```
db.restaurants.update (
```

```
  {"grades.grade" : {$not : {$eq : "C"}}},

  {$set : {"comment" : "acceptable"}}

);



WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Le résultat est identique. Il n'y aurait qu'un seul restaurant sans note 'C' ? Faites la requête find correspondante, vous verrez qu'il y en a 22 649 ! MongoDB interdit par défaut les modifications multiples avec un filtre. Cette opération pourrait être lourde et ralentir le système. Du coup, il faut explicitement dire que les modifications peuvent être multiples :

Unset

```
db.restaurants.update (

  {"grades.grade" : {$not : {$eq : "C"}}},

  {$set : {"comment" : "acceptable"}},

  {"multi" : true}
```

```
);
```

Update et javascript

Et si l'on cherchait à donner une note en fonction de l'ensemble des inspections effectuées par restaurant ? Cela devient compliqué avec l'opération **$set** (prend au mieux la valeur d'une clé avec un "**$**"). On va pour cela programmer un peu en *javascript* avec des fonctions itératives, et sauvegarder le résultat.

> Attention, cela peut être long car il faut exécuter la fonction pour chaque élément de la base, le modifier puis le sauvegarder.

Nous allons ajouter 3 points pour un rang A, 1 point pour B, et -1 pour C.

Unset

```
db.restaurants.find( {"grades.grade" : {$not : {$eq : "C"}}} ).forEach(

   function(restaurant){

      total = 0;

      for(i=0 ; i<restaurant.grades.length ; i++){

         if(restaurant.grades[i].grade == "A")         total += 3;

         else if(restaurant.grades[i].grade == "B")    total += 1;

         else                                          total -= 1;
```

```
      }

      restaurant.note = total;

      db.restaurants.save(restaurant);

   }

);
```

Le résultat ci-dessous montre que la meilleure note est obtenue par le "*Taco Veloz*" avec 24 points.

Unset

```
db.restaurants.find({},
{"name":1,"_id":0,"note":1,"borough":1}).sort({"note":-1});
```

```
{ "borough" : "Queens", "name" : "Taco Veloz", "note" : 24 }

{ "borough" : "Manhattan", "name" : "Gemini Diner", "note" : 22 }

{ "borough" : "Manhattan", "name" : "Au Za'Atar", "note" : 22 }

{ "borough" : "Brooklyn", "name" : "Lucky 11 Bakery", "note" : 22 }
```

```
{ "borough" : "Queens", "name" : "Mcdonald'S", "note" : 21 }

{ "borough" : "Queens", "name" : "Rincon Salvadoreno Restaurant", "note" :
21 }

{ "borough" : "Manhattan", "name" : "Kelley & Ping", "note" : 21 }
```

Remove

Nous allons maintenant supprimer tous les restaurants qui ont une note de 0.

```
Unset
db.restaurants.remove(

  {"note":0},

  {"multi" : true}

);
```

Save

Pour finir, une insertion (en dehors des possibilités d'importation avec mongoimport) s'effectue avec la fonction "*save*" :

```
Unset
```

```
db.restaurants.save({"_id" : 1, "test" : 1});
```

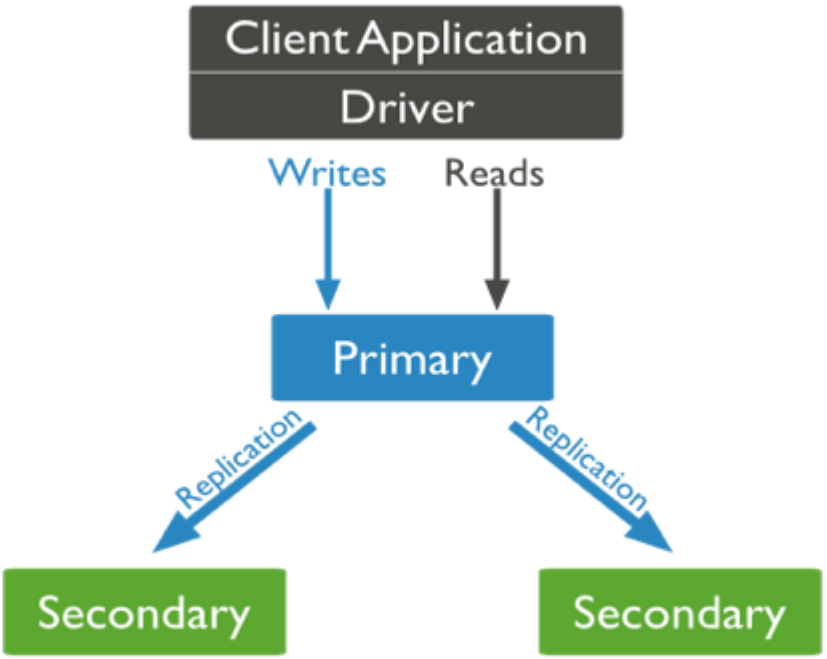
Dans le cas où un document de la collection aurait le même identifiant “_id”, le serait automatiquement écrasé par la nouvelle valeur.

Protégez-vous des pannes avec les ReplicaSet

L’architecture du ReplicaSet

Nous avons pu voir comment interroger des données, mais nous n’avons pas encore attaqué le problème de la tolérance aux pannes. MongoDB utilise pour cela une architecture basée sur le principe « maître/esclave ».

Le serveur primaire *"Primary",* à qui toutes les requêtes sont envoyées (lecture/écriture), va s’occuper de gérer la cohérence des données. Ainsi, lors d’une mise à jour, une réplication est effectuée sur les serveurs secondaires "*Secondary*".



Architecture d'un ReplicaSet

Toutefois, si l’on reste sur une architecture de la sorte, nous ne sommes pas tolérant aux pannes puisque le serveur primaire reste un point critique. Pour cela, nous allons rajouter un arbitre "*arbiter*", dont la tâche est de vérifier l’état du réseau. Ainsi, dès que le primary tombe, un système de vote va permettre d’élire le secondary qui

remplacera le serveur tombé en panne. Nous sommes donc tranquilles : cette architecture tient tant qu'au moins un serveur tourne. C'est ce que l'on appelle un **ReplicaSet**.

Un *ReplicaSet* doit contenir au minimum 3 serveurs (1 Primary et 2 Secondary) pour garantir un minimum de tolérance aux pannes. Un ReplicaSet peut contenir jusqu'à 50 serveurs ; toutefois lorsqu'un vote a lieu pour élire le nouveau Primary, au maximum 7 peuvent participer à cette élection (les premiers ayant répondu à l'Arbiter).

> **Stratégie de déploiement**
> C'est à vous de définir la stratégie de déploiement des serveurs physiques en fonction de votre architecture réseau. Il est préférable de mettre deux serveurs sur un même rack (remplacement du *primary*), dans le même *datacenter*, et au moins un dans un autre *datacenter* pour garantir la non perte de données.

> Nous pourrons par ailleurs associer des niveaux de priorité à chaque serveur dans le cas d'un vote.

Comment instancier un ReplicaSet ?

Nous allons simuler un ReplicaSet localement. Pour cela, les paramètres de chaque serveur doivent être définis :

1. Un nom de ReplicaSet : **rs0**
2. Un port d'écoute pour chaque serveur : **27018** (à incrémenter)
3. Créer un répertoire dédié : **/data/R0S1** (ReplicaSet 0/Serveur 1) - Sous *Windows,* utilisez le répertoire C:\data\R0S1 (à remplacer par la suite)
4. Ouvrir une console et aller dans le répertoire de mongodb (dénommé ici **$MONGO/bin**)
5. Lancer le serveur :`mongod --replSet rs0 --port 27018 --dbpath /data/R0S1`

Cette opération devra être répétée pour chaque serveur du *ReplicaSet*. Il faut maintenant initier celui-ci. En mode console (connexion sur le port 27018).

```
$MONGO/mongo --port 27018
```

Unset

```
rs.initiate ();
```

```
{
    "info2" : "no configuration specified. Using a default configuration for the set",
    "me" : "local:27018",
    "ok" : 1
}
```

Le ReplicaSet a été initié. Vous pouvez d’ailleurs connaître le nom réseau de votre machine dans la clé "me" (ici : **local**). Regardons la configuration du *ReplicaSet* :

Unset

```
rs.conf();

{
    "_id" : "rs0",
    "version" : 1,
```

```
    "protocolVersion" : NumberLong(1),
    "members" : [
        {
            "_id" : 0,
            "host" : "local:27018",
            "arbiterOnly" : false,
            "buildIndexes" : true,
            "hidden" : false,
            "priority" : 1,
            "tags" : {},
            "slaveDelay" : NumberLong(0),
            "votes" : 1
        }
```

```
    ],
    "settings" : {
        "chainingAllowed" : true,
        "heartbeatIntervalMillis" : 2000,
        "heartbeatTimeoutSecs" : 10,
        "electionTimeoutMillis" : 10000,
        "catchUpTimeoutMillis" : 2000,
        "getLastErrorModes" : {},
        "getLastErrorDefaults" : {"w" : 1,"wtimeout" : 0},
        "replicaSetId" : ObjectId("595dea41036cb3d77737ca61")
    }
}
```

Notre serveur est le premier membre du ReplicaSet "**rs0**", il a récupéré 1 voix (la sienne) pour devenir *PRIMARY*, toutefois, sans arbitre. Vous pouvez d'ailleurs constater dans votre console que le serveur est bien primary pour rs0 :

```
rs0:PRIMARY >
```

Maintenant, ajoutons deux autres membres au *ReplicaSet* ; reprendre les étapes 1 à 5 ci-dessus (répertoires, ports, 1 console par serveur). Puis :

```
Unset

rs.add("local:27019");

rs.add("local:27020");
```

Définir l'arbitre

Pour départager les votes (nombre pair de serveur par exemple), un arbitre sera nécessaire pour désigner le *PRIMARY*. Pour ce faire :

Créons le répertoire **/data/arb** (C:\data\arb sous Windows)
Exécutons un arbitre en spécifiant le port **30000** : `mongod --port 30000 --dbpath /data/arb --replSet rs0`
Ajoutons l'arbitre dans la console d'administration :

```
Unset

rs.addArb("local:30000")
```

Pour consulter le statut actuel du ReplicaSet, il suffit de faire l'instruction suivante :

```
Unset

rs.status()
```

```
{
    "set" : "rs0",
    "date" : ISODate("2017-07-06T13:36:41.283Z"),
    "myState" : 2,
    "term" : NumberLong(2),
    "syncingTo" : "local:27020",
    "heartbeatIntervalMillis" : NumberLong(2000),
    "optimes" : {
        "lastCommittedOpTime" : {
            "ts" : Timestamp(1499348198, 1),
            "t" : NumberLong(2)
        },
        "appliedOpTime" : {
```

```
        "ts" : Timestamp(1499348198, 1),
        "t" : NumberLong(2)
    },
    "durableOpTime" : {
        "ts" : Timestamp(1499348198, 1),
        "t" : NumberLong(2)
    }
},
"members" : [
    {
        "_id" : 0,
        "name" : "local:27018",
        "health" : 1,
```

```
        "state" : 2,
        "stateStr" : "SECONDARY",
        "uptime" : 21186,
        "optime" : {
            "ts" : Timestamp(1499348198, 1),
            "t" : NumberLong(2)
        },
        "optimeDate" : ISODate("2017-07-06T13:36:38Z"),
        "syncingTo" : "local:27020",
        "configVersion" : 3,
        "self" : true
    },
    {
```

```
        "_id" : 1,
        "name" : "local:27019",
        "health" : 1,
        "state" : 2,
        "stateStr" : "SECONDARY",
        "uptime" : 5625,
        "optime" : {
            "ts" : Timestamp(1499348198, 1),
            "t" : NumberLong(2)
        },
        "optimeDurable" : {
            "ts" : Timestamp(1499348198, 1),
            "t" : NumberLong(2)
```

```
        },
        "optimeDate" : ISODate("2017-07-06T13:36:38Z"),
        "optimeDurableDate" : ISODate("2017-07-06T13:36:38Z"),
        "lastHeartbeat" : ISODate("2017-07-06T13:36:40.037Z"),
        "lastHeartbeatRecv" : ISODate("2017-07-06T13:36:41.030Z"),
        "pingMs" : NumberLong(0),
        "syncingTo" : "local:27020",
        "configVersion" : 3
    },
    {
        "_id" : 2,
        "name" : "local:27020",
        "health" : 1,
```

```
        "state" : 1,
        "stateStr" : "PRIMARY",
        "uptime" : 5625,
        "optime" : {
            "ts" : Timestamp(1499348198, 1),
            "t" : NumberLong(2)
        },
        "optimeDurable" : {
            "ts" : Timestamp(1499348198, 1),
            "t" : NumberLong(2)
        },
        "optimeDate" : ISODate("2017-07-06T13:36:38Z"),
        "optimeDurableDate" : ISODate("2017-07-06T13:36:38Z"),
```

```
        "lastHeartbeat" : ISODate("2017-07-06T13:36:40.987Z"),
        "lastHeartbeatRecv" : ISODate("2017-07-06T13:36:39.473Z"),
        "pingMs" : NumberLong(0),
        "electionTime" : Timestamp(1499342576, 1),
        "electionDate" : ISODate("2017-07-06T12:02:56Z"),
        "configVersion" : 3
    },
    {
        "_id" : 3,
        "name" : "local:30000",
        "health" : 1,
        "state" : 7,
        "stateStr" : "ARBITER",
```

```
        "uptime" : 11,
        "lastHeartbeat" : ISODate("2017-07-06T13:36:41.309Z"),
        "lastHeartbeatRecv" : ISODate("2017-07-06T13:36:41.514Z"),
        "pingMs" : NumberLong(0),
        "configVersion" : 3
    }
  ],
  "ok" : 1
}
```

Comme on peut le constater, le *PRIMARY* peut changer. Dans le *ReplicaSet* ci-dessus, le serveur 3 (_id = 2, state = 1, port 27020) est devant le serveur maître (state = 2).

Tester la réplication

Pour tester la tolérance aux pannes, importons notre jeu de données (*mongoimport*) comme dans le premier chapitre, en utilisant le port du serveur *PRIMARY* (ici 27020). Vérifions l'existence des données avec une requête.

Simulons maintenant une panne du serveur *PRIMARY* en fermant le processus correspondant (dans le cas présent, le serveur ayant le port 27020). Vous pourrez alors constater qu'un nouveau serveur est élu *PRIMARY* (cette opération peut prendre un peu de temps).

Après connexion au nouveau *PRIMARY*, vous pourrez constater que votre requête retourne toujours le même résultat. Vous êtes tolérant aux pannes avec votre premier *ReplicaSet* !

Configuration

Il est possible de faciliter l'instanciation des *ReplicaSet* en créant un objet de connexion :

Unset

```
rsconf = {
        _id: "rs0",
        members: [
                {_id: 0, host: "local:27018"},
                {_id: 1, host: "local:27019"},
                {_id: 2, host: "local:27020"}
                ]
        };
rs.initiate (rs.conf) ;
```

En environnement de production, chaque serveur est unique avec un fichier de configuration associé. Il est format *YAML* et se trouve dans le répertoire *$MONGO/mongod.conf* (ou /etc/mongod.conf pour linux). Il se présente sous la forme suivante :

```
Unset

systemLog:

 destination: file

 path: "/var/log/mongodb/mongod.log"   #Fichier de log que nous verrons ensuite

 logAppend: true



storage:

 journal:

   enabled: true



```

```
processManagement:

  fork: true

#Permet de lancer le processus et de reprendre la main



net:

  bindIp: local

  port: 27018

#Adresse du serveur local et port d'écoute



replication:

  oplogSizeMB: 512

  replSetName: rs0

#Le nom de votre ReplicaSet
```

```
sharding:                #A voir dans le chapitre suivant

 clusterRole: shardsvr #C'est le mode par défaut. Sinon "configsvr"
```

Vous pourrez trouver l'ensemble des options du fichier de configuration ici.

Pour utiliser ce fichier de configuration :

```
mongod --config mongod.conf
```

Le fichier oPlog

Comment les données sont-elles répliquées ? Une collection particulière est définie automatiquement au niveau du serveur primaire : le fichier *oPlog*. Cette collection particulière va stocker toutes les opérations de mises à jour (journalisation), et les réplicas vont lire cette collection pour les appliquer directement sur leur image.

Par défaut, la réplication se fait en mode asynchrone ; les réplicas lisent en mode pull ce fichier oPlog. Toutefois, cela ne veut pas dire que notre système NoSQL est asynchrone. À partir du moment où nos données ne sont accessibles que sur le PRIMARY en permanence, la donnée est toujours cohérente. Toutefois, nous nous poserons la question autrement par la suite dans la haute disponibilité.

Vous pourrez retrouver cette collection dans la base de données « *local* » et la collection « *oplog.rs* ». Vous pouvez effectuer une requête pour voir le contenu.

Unset

```
use local;

db.oplog.rs.find().pretty();
```

On pourra y retrouver le timestamp d'insertion (*ts*), la version (*v*), le type d'opération (*"op" : "i"* pour insertion) et la collection cible (*"ns" : "new_york.restaurants"*).

Unset

```
{
  "ts" : Timestamp(1499753548, 21),
  "t" : NumberLong(9),
  "h" : NumberLong("-7247221887537468914"),
  "v" : 2,
  "op" : "i",
  "ns" : "new_york.restaurants",
  "o" : {
    "_id" : ObjectId("59646c4cb48094070fd0d7fc"),
    "address" : {
      "building" : "284",
      "coord" : {
```

```
            "type" : "Point",
            "coordinates" : [
                -73.9829239,
                40.6580753
            ]
        },
        "street" : "Prospect Park West",
        "zipcode" : "11215"
    },
    "borough" : "Brooklyn",
    "cuisine" : "American ",
    "grades" : [
        {
```

```
            "date" : ISODate("2014-11-19T00:00:00Z"),
            "grade" : "A",
            "score" : 11
        },
        {
            "date" : ISODate("2013-11-14T00:00:00Z"),
            "grade" : "A",
            "score" : 2
        },
        {
            "date" : ISODate("2012-12-05T00:00:00Z"),
            "grade" : "A",
            "score" : 13
```

```
        },
        {
            "date" : ISODate("2012-05-17T00:00:00Z"),
            "grade" : "A",
            "score" : 11
        }
    ],
    "name" : "The Movable Feast",
    "restaurant_id" : "40361606"
  }
}
```

Cette collection est donc régulièrement interrogée pour mettre à jour les réplicas. Il faut toutefois savoir que celle-ci a une taille limitée (*capped collection*). Elle garantit une lecture en continu des mises à jour. Toutefois, en fonction du taux de mises à

jour effectuées sur la base MongoDB, il peut être important d'augmenter la taille de l'*oPlog* pour ne pas perdre d'information. Sa taille par défaut dépend du type de moteur de stockage (par défaut WiredTiger) et du système d'exploitation : comme indiqué dans la documentation. Par défaut, il correspond à 5% de l'espace disque libre. L'option peut être mise à jour dans le fichier de configuration pour spécifier la taille maximale de ce fichier (par défaut 5Go). Nous avions mis précédemment 512Mo :

```
Unset

replication:

  oplogSizeMB: 512
```

Haute disponibilité

Afin de garantir un temps de réponse pour votre application, il est possible de modifier la gestion du ReplicaSet. Par défaut, toutes les requêtes de lectures sont effectuées sur le PRIMARY. Toutefois, cela alourdit sa charge ; il est également possible que celui-ci se trouve éloigné (physiquement sur le réseau) du client. Du coup, il est possible de router les requêtes de lectures sur les SECONDARY.

Pour ce faire, il faut définir les préférences de lecture avec les options suivantes :

- **Primary** : valeur par défaut, lecture sur le serveur d'écriture.
- **PrimaryPreferred** : si jamais le PRIMARY n'est plus disponible, les requêtes sont routées vers le SECONDARY en attendant que celui-ci réapparaisse (ou qu'un nouveau soit élu).
- **Secondary** : Routé uniquement sur les SECONDARY. Cela soulage la charge du serveur primaire. Toutefois, on perd alors en cohérence puisque les données sont répliquées de manière asynchrone.
- **Nearest** : Le serveur physique le plus proche sur le réseau (latence la plus faible) est interrogé directement par le client. Là non plus, la cohérence n'est pas garantie.

Le paramètre *localThresholdMS* du fichier de configuration permet de modifier la stratégie employée :

```
Unset

replication :

  localThresholdMS: <"primary"|"primaryPreferred"|"Secondary"| "Nearest">
```

Bien sûr, ces solutions font perdre en cohérence puisque les mises à jour n'attendent pas d'acquittement. Pour changer la stratégie d'accès aux données du *ReplicaSet*, le paramètre *readConcern* du fichier de configuration peut être modifié :

```
Unset

replication :

  readConcern: <"majority"|"local"|"linearizable">
```

Par défaut (*local*), les mises à jour sont asynchrones et seul le *PRIMARY* est consulté. Mais il est possible de vérifier que la majorité des réplicas soit synchronisée (*majority*), ou bien la lecture des données peut être ordonnée en fonction de l'instant d'écriture et de lecture (*linearizable*). Cet ordonnancement garantit la cohérence des lectures sur les réplicas mais réduit considérablement les performances.

Une dernière possibilité qu'offre un *ReplicaSet* est de favoriser le vote dans le cas d'une panne du *PRIMARY*. En effet, le système de vote peut prendre un certain temps. Vous pouvez accélérer ce vote en donnant une priorité entre les réplicas. Les réplicas qui ont la priorité la plus haute vont donc passer avant tous ceux qui ont une priorité inférieure. Pour ce faire, il suffit de modifier celles-ci avec :

```
Unset

cfg = rs.conf();
```

```
cfg.members[0].priority = 0.5;

cfg.members[1].priority = 2;

cfg.members[2].priority = 3;



rs.reconfig(cfg);
```

Le serveur « 2 » sera alors prioritaire lors d'une élection. Attention, la commande *reconfig* peut faire changer le PRIMARY !

Distribuez vos données avec MongoDB

L'architecture du sharding sous MongoDB

Jusqu'ici, nous avons vu les problèmes d'interrogation des données et de tolérances aux pannes. Il nous reste maintenant la question de la distribution pour le passage à l'échelle sous MongoDB. Cette distribution des blocs de données (*chunk*) appelée *sharding* est définie par un tri des données, contrairement à *HDFS* (placement physique) ou au *Consistent Hashing* (hachage sur un anneau). Ainsi, vous devez définir l'architecture physique de la distribution des serveurs appelée cluster.

La robustesse du *cluster* est gérée par 3 types de nœuds (serveurs) :

- Des routeurs : **mongos**
- Des serveurs de configuration : **Config Server**
- Des serveurs de données : **Shard**

La composition minimum d'un cluster est de 2 *mongos*, 3 *Config Servers* et 2 *shards*.

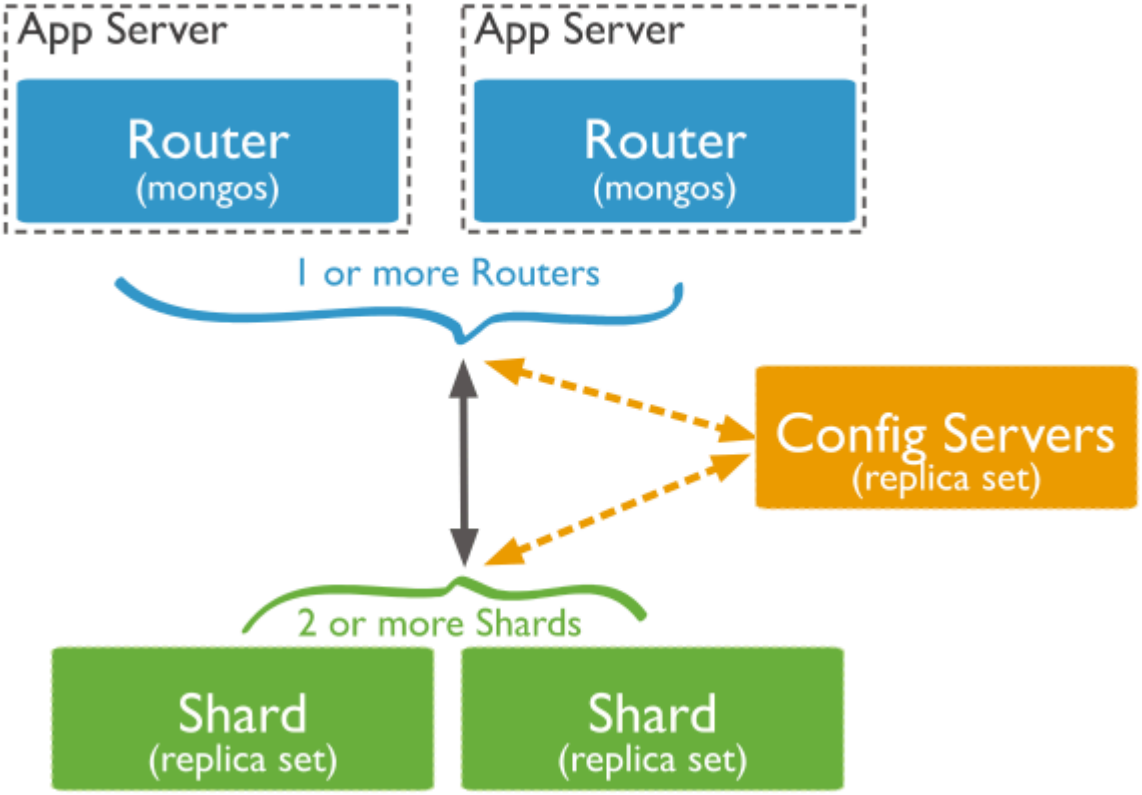
Un **mongos** s'occupe du routage des requêtes. Il stocke les informations sur l'arbre permettant de répartir les données (voir le chapitre sur le sharding). Ainsi, bien qu'il

soit un point central de l'architecture, le temps de traitement est rapide. Il faut minimum deux routeurs pour permettre de gérer la tolérance aux pannes de celui-ci.

Les **ConfigServers** s'occupent de la connaissance du réseau, aussi bien au niveau des routeurs que les *shards*. Ils vont ainsi stocker les informations sur l'arbre de routage utilisé par les routeurs ; permettant de synchroniser celui-ci et de l'intégrer à un nouveau routeur. Ils gèrent également les informations de répartition de charge sur les différents *shards*, et la structure des *ReplicaSets* (vu dans le chapitre précédent) permettant de rééquilibrer la charge des serveurs si la répartition des données est inégale. Les *ConfigServers* sont au nombre de 3 et organisés en *ReplicaSet* pour garantir l'intégrité du réseau. L'idéal est de les placer à des endroits clés du réseau pour des questions de pérennité du système (rack différents, proches des routeurs).

Les **shards** contiennent l'ensemble des données, les chunks. Ils peuvent contenir plusieurs chunks, mais pas forcément contigus (tri des données de l'arbre).

L'architecture de distribution d'un cluster est schématisée dans la figure ci-dessous :



Architecture du sharding sous MongoDB

Nous allons maintenant nous attaquer à la création de ce cluster. Pour son instanciation, il faudra suivre les étapes suivantes :

1. Lancer les **ConfigServer** en ReplicaSet (ici *configReplSet*)
2. Lancer chaque **Shard** en ReplicaSet (ici *sh1* et *sh2*)
3. Lancer un **mongos**

1. Connecter les ConfigServer
2. Connecter les shard
3. Lancer le sharding sur une collection

Création du serveur de configuration

Pour créer le cluster, il vous faut tout d'abord un *ConfigServer*. Bien sûr, il est nécessaire de (**1**) créer un répertoire dédié pour chaque serveur *mongod* (pas pour les *mongos*), (**2**) définir un port d'écoute à chacun. Les instructions suivantes sont à exécuter dans des consoles différentes :

```
mongod --configsvr --replSet configReplSet --port 27019
--dbpath /data/config1 mongod --configsvr --replSet
configReplSet --port 27020 --dbpath /data/config2
```

Nous avons donc lancé deux serveurs de configuration (**--configsvr**) en ReplicaSet (**configReplSet**). Dans une troisième console, vous pouvez vous connecter à un de ces serveurs et initier le ReplicaSet :

```
mongo --port 27019
```

```
Unset

rs.initiate();

rs.add("local:27020");
```

Création des shards

Ensuite, il est nécessaire de lancer les shard en ReplicaSet. Nous allons en créer deux pour tester la distribution. Pour chaque shard, le paramètre --shardsvr est nécessaire pour permettre son intégration.

```
mongod --shardsvr --replSet sh1 --port 27031 --dbpath
/data/sh1 mongod --shardsvr --replSet sh2 --port 27032
--dbpath /data/sh2 mongo --port 27031 --eval "rs.initiate()"
mongo --port 27032 --eval "rs.initiate()"
```

L'option **--eval** permet de faire passer la commande **rs.initiate()** directement au serveur puis de récupérer la main sur la console. Ici, cette commande permet d'initialiser le *ReplicaSet* de chaque *shard*.

Lancement du mongos et connexion des shards

Maintenant que nous avons des shards et des ConfigServers, nous pouvons nous attaquer au mongos (routeur).

```
mongos --configdb configReplSet/localhost:27019 --port 27017
```

Il est possible de rajouter chacun des serveurs du ReplicaSet, en les séparant par des virgules :

```
configReplSet/localhost:27019,localhost:27020,localhost:27021
```

Les *shards* peuvent alors être ajoutés les uns après les autres au niveau du mongos en mode console :

```
mongo --port 27017
```

Unset

```
sh.addShard( "sh1/localhost:27031");

sh.addShard( "sh2/localhost:27032");
```

Distribution de la base de données

Ça y est, l'architecture de distribution est mise en place. Il suffit maintenant de définir la collection que l'on veut distribuer. Pour cela, nous allons créer une base « testDB » et une collection « test ».

Unset

```
use testDB;

sh.enableSharding("testDB");

db.createCollection("test");
```

```
db.test.createIndex({"_id":1});

sh.shardCollection("testDB.test",{"_id":1});
```

La collection test est maintenant distribuée automatiquement sur nos deux shards, les documents sont placés dans les *chunks* en fonction de leur valeur d'identifiant "_id", un tri de ces valeurs est effectué pour déterminer le placement (cf sharding arborescent). Nous pouvons maintenant importer les données des restaurants dans cette base de test :

```
mongoimport --db testDB --collection test --port 27017
restaurants_NY.json mongo --port 27017 --eval "sh.status()"
```

Unset

```
--- Sharding Status ---

sharding version: {

"_id" : 1,

"minCompatibleVersion" : 5,

"currentVersion" : 6,

"clusterId" : ObjectId("5967a3f7695a5111518224c6")

}
```

```
shards:

{  "_id" : "sh1",  "host" : "rs0/local:27031",  "state" : 1 }

{  "_id" : "sh2",  "host" : "rs1/local:27032",  "state" : 1 }

active mongoses:

"3.4.3" : 1

autosplit:

Currently enabled: yes

balancer:

Currently enabled:  yes

Currently running:  no

      Balancer lock taken at Thu Jul 13 2017 18:46:47 GMT+0200 (CEST) by
ConfigServer:Balancer

Failed balancer rounds in last 5 attempts:  0
```

```
Migration Results for the last 24 hours:

      2 : Success

databases:

{  "_id" : "testDB",  "primary" : "rs0",  "partitioned" : true }

      testDB.test

            shard key: { "_id" : 1 }

            unique: false

            balancing: true

            chunks:

                  sh1     2

                  sh2     2

            { "_id" : { "$minKey" : 1 } } -->> { "_id" :
ObjectId("5967a455b6c0223b91eefbd1") } on : rs1 Timestamp(2, 0)

            { "_id" : ObjectId("5967a455b6c0223b91eefbd1") } -->> { "_id" :
ObjectId("5967a455b6c0223b91eeffba") } on : rs1 Timestamp(3, 0)
```

```
            { "_id" : ObjectId("5967a455b6c0223b91eeffba") } -->> { "_id" :
ObjectId("5967a455b6c0223b91ef01af") } on : rs0 Timestamp(3, 1)
```

```
            { "_id" : ObjectId("5967a455b6c0223b91ef01af") } -->> { "_id" :
{ "$maxKey" : 1 } } on : rs0 Timestamp(1, 4)
```

Nous pouvons constater dans la partie « **databases** » que la collection *testDB.test* est composée de 4 *chunks*, répartis uniformément sur les deux *shards*.

> *enableSharding* ne marche **que** sur une collection préalablement **vide**. Toute collection déjà existante ne peut être distribuée. Il faudra ainsi importer les données dans la collection « *shardée* ».

Stratégie de distribution

Nous avons vu comment créer une architecture de distribution et comment distribuer une collection. Maintenant, il serait intéressant de savoir comment choisir une stratégie de distribution appropriée à nos données, afin de fournir les meilleures performances pour nos requêtes.

Requêtes par intervalles

Jusqu'ici, nous nous sommes contentés de choisir l'identifiant « **_id** » pour distribuer les données. Par défaut, l'indexation est basée sur une index non-dense triant les données. De fait, les requêtes par intervalles de valeurs sont optimisées. Il est donc possible de choisir une autre clé pour les documents de la collection comme stratégie de distribution.

Pour profiter des requêtes par intervalles, nous pourrions créer une « clé de sharding » sur le code postal, la clé : *zipcode*. Ainsi, toute requête cherchant à retrouver les restaurants associés à ce code postal ou une plage de codes postaux seront efficaces.

Tout changement de stratégie de sharding impose la création d'une nouvelle collection. On ne peut changer la distribution d'une collection déjà « shardée ».

```
Unset

sh.shardCollection("testDB.test2", {"address.zipcode" : 1});
```

Requêtes par zones

Le *zipcode*, c'est bien, mais comment mieux maîtriser la répartition des restaurants sur les différents *chunks* ? En effet, la technique précédente ne permet pas de contrôler la répartition physique des *chunks* et les intervalles de valeurs de ceux-ci. Le *sharding* par zone permet d'associer à chaque *shard* une zone. Tous les restaurants de cette zone seront alors alloués à un serveur dédié. L'avantage est double :

1. **Optimiser** toutes **requêtes** liées à une **zone** (filtrage ou agrégation par quartier).
2. Définir un **serveurphysique** dans un *datacenter***proche** du client (zone géographique), correspondant à des requêtes de proximité.

Pour cela, il faut tout d'abord définir ces zones. Une zone est composée d'une plage de valeur et d'un « tag ». Nous pouvons choisir sur un intervalle de codes postaux correspondant aux villes :

```
Unset

sh.addTagRange("testDB.test2",

   {"address.zipcode": "10001"}, {"address.zipcode":  "11240"},

   "NYC")

sh.addTagRange("testDB.test2",
```

```
    {"address.zipcode": "94102"}, {"address.zipcode":  "94135"},

    "SFO")
```

L’avantage est de pouvoir associer des *shards* physiquement proches des utilisateurs dans chaque ville pour réduire la latence du réseau. Cela peut avoir également un intérêt si l’on souhaite stocker des données dans un cluster sécurisé.

Attention toutefois à la surcharge d’un *shard* suite à une zone mal dimensionnée. Nous pouvons découper une zone sur deux tags.

Unset

```
sh.addTagRange("testDB.test2",

    {"address.zipcode": "10001"}, {"address.zipcode": "10281"},

    "NYC-Manhattan")
```

```
sh.addTagRange("testDB.test2",
  {"address.zipcode": "11201"}, {"address.zipcode": "11240"},
  "NYC2-Brooklyn")
```

Pour information, la plage de valeurs inclut le min (10001), mais pas le max (10281).

Maintenant que les tags sont associés à des plages de valeurs, il suffit d'associer à chaque tag un shard. Un tag peut être associé à plusieurs shards.

Unset

```
sh.addShardTag("sh1", "NYC")
sh.addShardTag("sh2", "NYC")
sh.addShardTag("sh2", "SFO")
```

Automatiquement, les restaurants seront placés dans les *chunks* des *shards* désignés en fonction de leurs codes postaux. Il faudra toutefois rester vigilant à la répartition de charge et le dimensionnement des *shards*. Pour cela, vous pouvez consulter la répartition des *chunks* d'un tag dans le réseau :

```
Unset

use config;

db.shards.find({ "tags" : "NYC" });
```

Requêtes par catégories

Et si nous faisions des requêtes ciblées sur un certain type de valeurs comme le quartier (*borough*) ? Il n'y a dans ce cas pas de plages de valeurs possibles, uniquement des restaurants correspondant à un quartier donné. Un simple partitionnement par hachage suffirait amplement à la segmentation des données.

MongoDB offre la possibilité de faire un hachage (via la fonction *md5*) sur les valeurs de la clé de sharding. L'avantage est soit de pouvoir **contrôler le placement** de tous les documents partageant la même valeur, soit de faire une **répartition plus uniforme** des documents sur l'ensemble des shards.

```
Unset

sh.shardCollection( "testDB.test3", { "borough" : "hashed" } )
```

> Attention, le routeur n'est alors efficace que si l'on utilise des requêtes d'égalité sur le quartier. Les requêtes par intervalles ne permettent pas d'effectuer un routage sur les *shards*. De fait, la requête est transmise à tous les *shards* par *broadcast*.

Index et performances

Bien entendu, nous ne pouvons choisir qu'une seule stratégie de distribution pour optimiser nos requêtes. Mais MongoDB offre la possibilité d'interroger n'importe quelle clé. Comment éviter alors le *broadcast* lorsque la clé de *sharding* n'est pas utilisée ?

La collection « *testBD.test* » est distribuée sur l'identifiant des restaurants « *_id* ». Je souhaite maintenant exécuter la requête suivante :

Unset

```
db.test.find({"address.street" : "3 Avenue"}).explain()
```

```
{
  "queryPlanner" : {
    "mongosPlannerVersion" : 1,
    "winningPlan" : {
      "stage" : "SHARD_MERGE",
      "shards" : [
        {
          "shardName" : "sh1",
          "connectionString" : "sh1/local:27030",
          "serverInfo" : {
            "host" : "local",
```

```
        "port" : 27031,
        "version" : "3.4.3",
        "gitVersion" : "f07437fb5a6cca07c10bafa78365456eb1d6d5e1"
      },
      "plannerVersion" : 1,
      "namespace" : "testDB.test",
      "indexFilterSet" : false,
      "parsedQuery" : {
        "address.street" : {
          "$eq" : "3 Avenue"
        }
      },
      "winningPlan" : {
```

```
        "stage" : "SHARDING_FILTER",
        "inputStage" : {
          "stage" : "COLLSCAN",
          "filter" : {
            "address.street" : {"$eq" : "3 Avenue"}
          },
          "direction" : "forward"
        }
      },
      "rejectedPlans" : [ ]
    },
    {
      "shardName" : "sh2",
```

```
        "connectionString" : "sh2/local:27031",
        "serverInfo" : {
          "host" : "local",
          "port" : 27032,
          "version" : "3.4.3",
          "gitVersion" : "f07437fb5a6cca07c10bafa78365456eb1d6d5e1"
        },
        "plannerVersion" : 1,
        "namespace" : "testDB.test",
        "indexFilterSet" : false,
        "parsedQuery" : {
          "address.street" : {
            "$eq" : "3 Avenue"
```

```
        }
      },
      "winningPlan" : {
        "stage" : "SHARDING_FILTER",
        "inputStage" : {
          "stage" : "COLLSCAN",
          "filter" : {
            "address.street" : {
              "$eq" : "3 Avenue"
            }
          },
          "direction" : "forward"
        }
```

```
        },
        "rejectedPlans" : [ ]
    }
    ]
  }
  },
  "ok" : 1
}
```

La fonction "**.explain()**" appliquée à une requête permet de consulter le plan d'exécution généré. Ici, le plan « *WinningPlan* » possède une étape (stage) « *SHARD_MERGE* » qui effectue la fusion des résultats provenant de deux « *COLLSCAN* » (« *Collection Scan* »), en l'occurrence un scan complet de la collection pour chaque *shard*. Cela correspond à une lecture complète du contenu des chunks, ce qui n'est pas très efficace.

Nous allons créer un index sur la rue :

Unset

```
db.test.createIndex({"address.street" : 1})
```

Après avoir testé à nouveau la requête, l'index est utilisé car l'étage passe en « *IXSCAN* ». Ce qui va permettre d'accéder directement aux restaurants de la 3° avenue sans avoir à parcourir toute la base de données. Bien sûr, comme ce n'est pas la clé de *sharding*, tous les *shards* effectuent cette recherche. Même si un seul restaurant correspond à un critère de recherche, tous les *shards* parcourent l'index. Il n'y a pas d'index global, uniquement des indexes locaux à chaque *shard*. Toutefois, les performances seront améliorées.

Si l'on souhaite profiter des avantages d'un index dans le cadre d'une requête « *aggregate* », il faut que le premier critère de recherche soit un filtre sur la clé utilisée dans l'index. Dans notre cas, seul un opérateur de type **$match** contenant « *address.street* » permettrait l'utilisation de l'index (idem pour la clé de *sharding*).

Pour consulter le plan d'exécution d'un « *aggregate* », le paramètre *{explain:true}* doit être ajouté après la séquence d'opérations :

Unset

```
db.test.aggregate([

  {$match:{"address.street" : "3 avenue"}},

  {$group:{_id:"$borough", total : { $sum : 1}}}

],
```

```
{explain:true}
```

```
};
```

Entraînez-vous à créer et à interroger une base de données MongoDB

Entraînez-vous à créer et à interroger une base de données MongoDB

À vous de jouer

> Pour vous entraîner, réalisez cet exercice étape par étape. Une fois terminé, vous pouvez comparer votre travail avec les pistes que je vous propose.

Nous allons intégrer une base de données touristique sur la ville de Paris. Téléchargez le fichier "tour-Pedia_paris.json.zip" à importer. Cette base de données contient des **lieux** de différentes catégories : des points d'intérêt (*POI*), des restaurants (*restaurant*), des attractions (*attraction*), des hôtels (*accommodation*).

La structure des documents que vous pourrez consulter avec la commande "*findOne*()" est présenté ci-dessous. Elle nous servira de base pour l'interrogation.

Unset


```
{
  "_id" : 83292,
  "contact" : {
    "website" : "http://www.trocaderolatour.com",
    "GooglePlaces" :
"https://plus.google.com/107754700607079935569/about?hl=en-US"
  },
  "name" : "Best Western Premier Trocadero La Tour",
  "location" : {
    "city" : "Paris",
```

"coord" : {"coordinates" : [2.2795155644417,48.858311118724],"type" : "Point"},

"address" : "Paris, France 5 bis, rue Massenet, 16. Trocadéro - Passy, 75016 Paris"

},

"category" : "accommodation",

"description" : " Situé à 15 minutes à pied de la tour Eiffel, le Best Western Premier Trocadero La Tour bénéficie d'un emplacement idéal pour découvrir Paris. Il abrite un bar lambrissé doté de fauteuils en cuir et un patio.",

"services" : [

"jardin",

"terrasse",

"journaux",

```
    "bar",

    "petit-déjeuner en chambre",

    "réception ouverte 24h 24",

    "enregistrement et règlement rapides",

    "bagagerie",

    "service d'étage",

    "salles de réunions banquets",

    "centre d'affaires",

    "garde d'enfants",

    "blanchisserie",
```

```
    "chambres non-fumeurs"

  ],

  "reviews" : [

    {

      "wordsCount" : 30,

      "rating" : 0,

      "language" : "en",

      "source" : "Foursquare",

      "text" : "Nice beds, rooms andstaff. Perfect central location. Breakfast is very expensive for a contenintal breakfast, however many bakeries and restaurants in the area. Will stay here again my next visit.",

      "time" : "2010-09-30"
```

```
    }

  ]

}
```

Pour chacune des questions suivantes, veuillez fournir la commande associée et non le résultat.

1. **Importez** le jeu de données tour-pedia.json dans une base de données “**tourPedia**” avec une collection “**paris**” ;
2. **Filtrez** les lieux par type “*accommodation*” et service “*blanchisserie*” ;
3. **Projetez** les adresses des lieux de type "accommodation" ;
4. **Filtrez les listes** de commentaires (*reviews*) des lieux, pour lesquelles au moins un commentaire (*reviews*) est écrit en anglais (*en*) et a une note (*rating*) supérieure à 3 (attention, **LE** commentaire en anglais doit avoir un rating de 3 ou plus) ;
5. **Groupez** les lieux par catégorie et comptez les ;
6. Créez un **pipeline d’agrégation** pour les lieux de catégorie "*accommodation*", et donnez le nombre de lieux par valeur de "*services*".

Vérifiez-bien que vous avez les éléments suivants :

- La commande “*mongoimport*” est présente avec les paramètres de bases de données “**--db tourPedia”**, collection “**--collection paris**” et fichier à importer “**tour-pedia.json**”

Unset

```
mongoimport --db tourPedia --collection paris tourPedia_paris.json
```

- Le document JSON requête (premier paramètre de la fonction "find()") contient les deux critères demandés "*services*" et "*category*".

Unset

```
db.paris.find ({"services":"blanchisserie", "category":"accommodation"})
```

- Le document JSON présent en deuxième paramètre de la fonction "find()" contient la clé du document à projeter "location.address". Étant donné que la clé "address" est imbriquée dans "location", la projection est la combinaison des deux, séparés par un point "."

Unset

```
db.paris.find ({"category" : "accommodation"}, {"location.address":1})
```

- La liste de documents "*reviews*" est filtrée. Comme les critères sur les clés "*language*" et "*rating*" sont utilisés sur le même document de la liste "*reviews*", l'opérateur **$elemMatch** est utilisé et imbrique les deux critères.

Unset

```
db.paris.find ({"reviews" : {$elemMatch : {"language":"en", "rating" : {$gt
: 3}}}})
```

- La fonction "*aggregate()*" est utilisée avec une liste d'opérateurs "[ ]". L'opérateur $**group**, est appliqué sur la catégorie : **_id : "$category"** (le "$" est obligatoire). Pour compter le résultat **{$sum:1}** est associé à une clé ("*tot*" n'est pas obligatoire, n'importe quel nom peut être utilisé

Unset

```
db.paris.aggregate( [ {$group : {_id:"$category", "tot":{$sum:1}} } ] ) ;
```

- La liste d'opérateurs de la fonction "*aggregate()*" contient trois opérateurs : **$match**, **$unwind** et **$group**. L'ordre *$match/$unwind* n'est pas indispensable ici. Par contre, l'opérateur *$group* est le dernier opérateur de

la liste. Les opérateurs *$unwind et $group* doivent contiennent "**$services**" (sans oublier le $).

Unset

```
db.paris.aggregate ([

  {$match :  {"category" : "accommodation"} },

  {$unwind : "$services" },

  {$group : {_id:"$services", "tot":{$sum:1}} }

])
```